THE AGENTIC AI ENGINEERING BOOTCAMP

Capstone Project Report – By: Marwan Aziz

Project 3: Autonomous Equity Research Agent

---

1. Introduction and Problem Statement

1.1 The Problem

Equity research is one of the most information-intensive workflows in finance. A junior analyst at an investment firm routinely spends two to three days on a single research report: reading SEC filings to identify risk factors, pulling financial data from terminals such as Bloomberg or Refinitiv, identifying comparable peer companies and extracting their valuation multiples, synthesizing all of this into a coherent investment thesis, and finally writing a structured report with a buy, hold, or sell recommendation. The work is critical to portfolio decisions, yet the majority of it involves gathering, formatting, and synthesizing information that is already publicly available.

This creates a meaningful opportunity for automation. Most of the analyst's time is consumed not by judgment, but by data retrieval and structured synthesis. Tasks that a well-designed multi-agent system can perform in under two minutes.

1.2 The Solution

This project implements an Autonomous Equity Research Agent: a full-stack web application where a user enters any publicly traded stock ticker and receives a professional-grade equity research report within approximately 90 seconds. The report includes the company's current price and key financial metrics, a peer comparison table with valuation multiples for three comparable companies, the top risk factors extracted from SEC 10-K filings and financial news, a buy, hold, or sell investment signal with a written rationale, and a full multi-section research narrative covering the company overview, financial analysis, peer comparison, and investment recommendation.

The application is built on a five-agent pipeline that mirrors the workflow of a real analyst team: a Financial Data Agent, a Comparable Companies Agent, a Risk Analysis Agent, an Analyst Agent, and a Research Writer Agent. Each agent specializes in one stage of the research process. The entire pipeline streams its progress to the user in real time, with each agent's status updated as it completes.

1.3 Real-World Context

This project addresses the same problem domain as Bloomberg Terminal ($25,000 per year per seat), Koyfin, Simply Wall St, and Finviz. Unlike those platforms, which are tools requiring human analysis, this system produces a complete written research report autonomously. It demonstrates how multi-agent AI systems can compress days of professional knowledge work into minutes.

---

2. The Problem-First Approach

The project followed the six-step Problem-First Method described in the bootcamp curriculum.

Step 1: Understand the Problem

The research question was: which parts of equity research can be fully automated, and what is the minimum output required for a report to be actionable? The answer guided the system design. Price and financial metrics are entirely retrievable from data sources. Peer identification requires judgment but can be guided by web search. Risk factors exist verbatim in SEC filings. The investment signal requires synthesis and reasoning. This is the highest-order task in the pipeline. Writing the final report is a formatting and prose task. Each of these maps cleanly to a specialized agent.

The definition of "done" was taken directly from the project brief: current price and key financials retrieved; at least three comparable companies identified with valuation multiples; a buy, hold, or sell signal with reasoning; results displayed in a readable format; works in a live demo.

Step 2: Break It Into Pieces

The project was decomposed into five major parts:

1. **Financial data retrieval:** fetching the current price, market cap, P/E ratio, revenue, EBITDA, margins, and related metrics for the subject company.
2. **Comparable company identification:** finding three peer companies with similar business models and extracting their valuation multiples for a comparison table.
3. **Risk factor extraction:** identifying the top five to eight material risks from SEC filings and recent financial news.
4. **Analyst synthesis:** comparing the subject company to its peers, assessing the valuation, and producing a buy, hold, or sell signal with explicit reasoning.
5. **Report writing:** formatting all findings into a structured, professional equity research report with clearly labeled sections.

A sixth parallel track covered the frontend: a streaming web interface that shows each agent's progress in real time and renders the final report in a structured, readable layout.

Step 3: Write the Prompts

Each agent was given a Context-Instruction-Input-Output (CIIO) prompt. The full prompts are documented in Section 4. Each prompt was designed so that its output is the natural input to the next agent, creating a clean sequential pipeline.

Step 4: Review Together

The prompt chain was reviewed for consistency before implementation. The following are the dependency checks:

1) does the Comps Agent receive enough context from the Financial Data Agent to identify appropriate peers?

2) Does the Analyst Agent receive all three data sources (financials, comps, risk) in a structured format?

3) Does the Writer Agent receive the analyst's signal and reasoning verbatim so it can carry them forward

without modification?

These dependencies were explicitly encoded in the pipeline's prompt-building functions.

Step 5: Build

Claude Code was used to implement the entire application. The implementation proceeded in layers: backend scaffolding, Pydantic models, tools, agents, pipeline orchestration, and frontend components. Each layer was verified before the next was built.

Step 6: Demo

The application was tested with AAPL (large-cap technology), NVDA (high-growth AI semiconductor), JPM (financial services), and COST (retail, stable business). Those were the test tickers specified in the project brief. All four produced complete, well-structured reports with valid buy, hold, or sell signals.

---

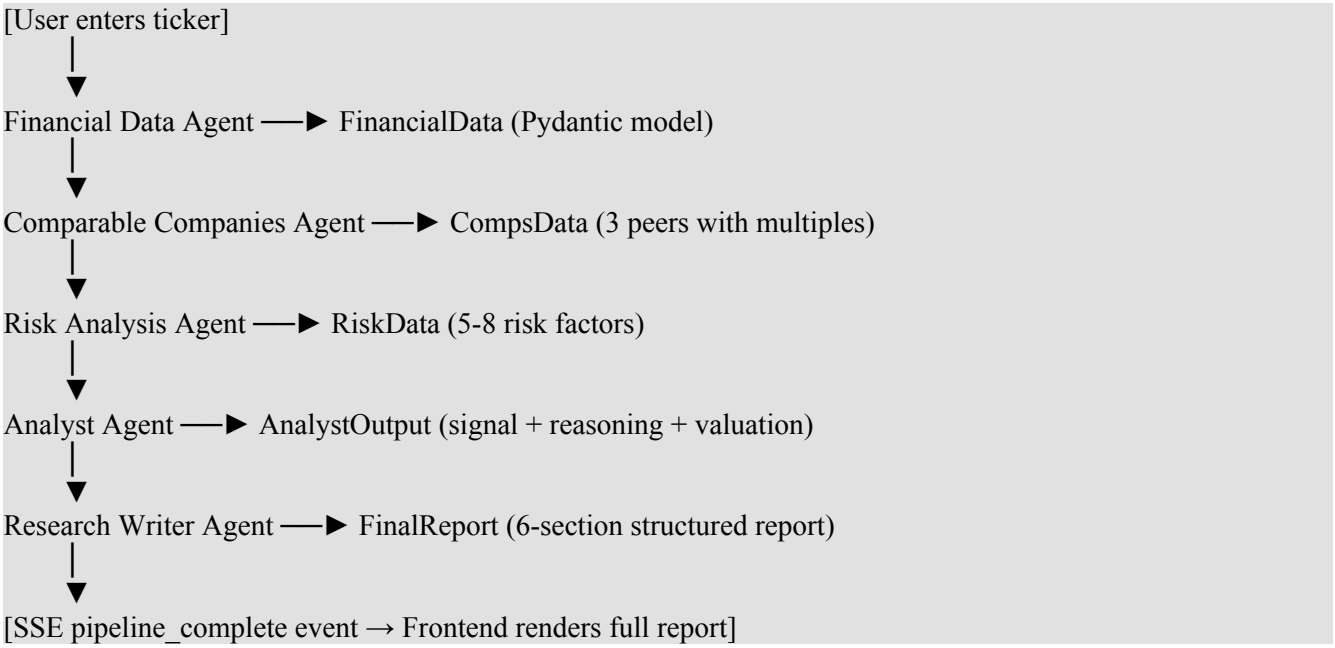
3. System Architecture

3.1 Technology Stack

Layer	Technology	Rationale
Agent Framework	OpenAI Agents SDK 0.0.19	Matches Week 2 bootcamp module; provides structured output via Pydantic, tool-calling, and async Runner
LLM (data agents)	GPT-4.1-mini	Fast, low-cost, sufficient for structured data extraction and peer identification
LLM (analyst + writer)	GPT-4.1-mini	Balance of reasoning quality and speed for the synthesis and writing steps
Financial Data	Tavily Web Search API	Reliable alternative to Yahoo Finance, which rate-limits heavily in automated contexts
SEC Filings	edgartools + Tavily fallback	edgartools provides structured 10-K access; Tavily is the fallback for companies without U.S. SEC filings
Backend API	FastAPI + Uvicorn	Async-native Python framework; supports Server-Sent Events natively via StreamingResponse
Frontend	Next.js 16 + TypeScript + Tailwind CSS	App Router with streaming API routes; matches the Week 3 Calorie Tracker stack
Streaming	Server-Sent Events (SSE)	Unidirectional server-to-client streaming; simpler than WebSockets and natively supported in browsers via EventSource

3.2 Agent Pipeline

The five agents run sequentially. Each agent's output is passed as structured input to the next:



3.3 Streaming Architecture

The backend exposes a single endpoint: GET /api/research/stream/{ticker}. This returns a StreamingResponse with Content-Type: text/event-stream. As each agent starts and completes, the pipeline yields a named SSE event with a JSON payload. The frontend opens an EventSource connection and updates the UI in real time as events arrive.

A critical implementation detail: Next.js's built-in rewrites proxy buffers streaming responses, delivering all events at once when the stream closes. This was resolved by implementing a Next.js App Router API route (app/api/research/stream/[ticker]/route.ts) that passes the upstream ReadableStream directly as the Response body, bypassing any buffering layer.

A second streaming challenge: the Research Writer Agent can take 60 to 90 seconds to generate a complete multi-section report. During this time, no SSE events are sent, which can cause proxy and browser connections to time out. This was resolved by running the writer agent inside an asyncio.shield loop that yields SSE comment lines (: keepalive) every 12 seconds, keeping the connection alive without sending substantive data.

3.4 Data Architecture

All agent outputs are typed using Pydantic BaseModel classes. The key models are:

- **FinancialData**: 20+ fields covering price, market cap, valuation multiples, income metrics, balance sheet ratios, and analyst consensus.
- **CompsData**: A list of CompanyMetric objects (ticker, name, market cap, P/E, EV/EBITDA, revenue, margin, P/B) plus a search rationale.
- **RiskData**: A list of risk factor strings and the filing year.

- **AnalystOutput**: Signal (BUY/HOLD/SELL), reasoning, valuation summary, relative valuation, key strengths, key risks, and price target rationale.
- **FinalReport**: Six narrative sections plus the signal, reasoning, strengths, risks, and an executive summary.

---

4. Prompts Used in Claude Code

Each agent is defined with a CIIO-structured system prompt. The following are the actual prompts used in the implementation.

Prompt 1: Financial Data Agent

Context: A financial data specialist agent within a multi-agent equity research pipeline. The agent has access to a Tavily web search tool.

Instruction: Run four targeted web searches for the subject ticker:

1. Current price, 52-week range, and analyst price target.
2. P/E ratio, forward P/E, price-to-book, EPS, beta, and dividend yield.
3. Annual revenue, EBITDA, and market cap from income statement data.
4. Profit margin, return on equity, and debt-to-equity ratio from balance sheet data.

Extract numeric values from the results, convert units correctly ($242B → 242000000000, 55% → 0.55), and populate a FinancialData Pydantic model. Set fields to null only if the value genuinely cannot be found. Never fabricate numbers.

Input: Stock ticker symbol (string), Tavily API key (environment variable).

Output: A validated FinancialData object with all available fields populated.

Prompt 2: Comparable Companies Agent

Context: A buy-side equity analyst specializing in peer group analysis. The agent has access only to the Tavily web search tool.

Instruction: Run two Tavily searches: one to identify peer companies by sector and business model similarity, and one per peer to find their financial metrics from financial news sites. Select exactly three peers with similar business models and scale. Extract market cap, P/E, EV/EBITDA, revenue, profit margin, and price-to-book for each peer. Return a CompsData object with the three companies and a one-to-two-sentence rationale for why they were selected.

Input: Subject ticker, company name, sector, industry, market cap, P/E, and revenue from the FinancialData output.

Output: A CompsData object with exactly three comparable companies and their financial metrics.

Prompt 3: Risk Analysis Agent

Context: A risk analyst specializing in SEC filings and financial disclosures. The agent has access to the edgartools library (for structured 10-K access) and Tavily web search (as a fallback).

Instruction: Call the get_10k_risk_factors tool for the subject ticker to fetch Item 1A from the most recent 10-K filing. If the SEC filing is unavailable (e.g., foreign-listed ADRs), fall back to Tavily search for recent risk factor coverage. Synthesize the top five to eight most material risk factors as complete, specific sentences — not headings. Focus on competitive risks, regulatory risks, macroeconomic risks, execution risks, and financial risks.

Input: Stock ticker symbol.

Output: A RiskData object with the ticker, company name, five to eight risk factor statements, and the filing year.

Prompt 4: Analyst Agent

Context: A senior equity research analyst at a top investment bank. No tools — this agent reasons over the structured data provided.

Instruction: Review the financial data, peer comparison, and risk factors. Produce a BUY, HOLD, or SELL signal using a three-part framework: (1) relative valuation — compare the subject's P/E, EV/EBITDA, P/B, and margins to the peer group, identifying whether the premium or discount is justified; (2) absolute valuation — assess the stock against its 52-week range and analyst consensus target; (3) quality and risk — consider ROE, debt-to-equity, and the top risk factors to adjust the signal for downside scenarios. Cite specific numbers from the data provided.

Input: Structured text block containing FinancialData, CompsData, and RiskData.

Output: An AnalystOutput object with the signal, reasoning, valuation summary, relative valuation narrative, key strengths list, key risks list, and price target rationale.

Prompt 5: Research Writer Agent

Context: An equity research writer at a top investment bank. No tools — this agent formats and writes.

Instruction: Write a six-section professional equity research report using the provided financial data, peer comps, risk factors, and analyst output:

1. Company Overview (two to three paragraphs on business model, market position, and recent developments).
2. Financial Overview (two to three paragraphs on valuation metrics, revenue and earnings trends, and balance sheet).
3. Peer Comparison (two to three paragraphs comparing multiples to the peer group).
4. Risk Summary (one to two paragraphs synthesizing the top risks).
5. Recommendation (two to three paragraphs restating the investment thesis and bull/bear triggers).
6. Executive Summary (three to five sentences capturing the entire thesis for a portfolio manager).

Carry the signal, signal reasoning, key strengths, and key risks verbatim from the analyst output. Write in professional, precise, data-anchored prose. Cite specific numbers.

Input: Structured text block with all five data sources (financials, comps, risks, analyst signal, analyst reasoning).

Output: A FinalReport Pydantic object with all six sections populated.

5. Justification of Methods and LLMs

5.1 OpenAI Agents SDK

The OpenAI Agents SDK was chosen for three reasons. First, it was the agent framework used in the Week 2 bootcamp module, providing a known starting pattern (the Researcher-Analyst-Writer pipeline) that could be adapted for finance. Second, it provides structured output enforcement via Pydantic — every agent declares an output_type, and the SDK guarantees the model's response matches that schema before returning control to the pipeline. This eliminates the need for manual JSON parsing or output validation. Third, it handles tool calling transparently: the agent decides when to call Tavily or edgartools, how many times, and with what parameters, without requiring orchestration code for each possible tool call sequence.

5.2 GPT-4.1-mini for Data Agents

The Financial Data, Comparable Companies, Risk, and Writer agents use GPT-4.1-mini. This model is well-suited for structured extraction tasks — it can parse numeric values from search result text, convert units, and populate a Pydantic schema reliably. It is faster and lower-cost than GPT-4.1, which matters for a pipeline that calls the model five times per research request.

5.3 Tavily as the Primary Data Source

The original project specification called for yfinance (Yahoo Finance) for financial data. During implementation, Yahoo Finance's API was found to be heavily rate-limited in automated environments: the crumb authentication endpoint returns HTTP 429 responses under load, and yfinance stores the error response body as the crumb value, permanently poisoning subsequent calls. This is a known and recurring issue with yfinance in production contexts.

Tavily was chosen as the replacement because: it is already required for the web search component; it provides reliable access to financial data aggregated from sites such as CNN Money, Macrotrends, Stock Analysis, and others; and it returns structured answer summaries that GPT-4.1-mini can parse accurately. The Financial Data Agent runs four targeted searches: price/market data, valuation multiples, income statement metrics, and balance sheet metrics. Those searches together cover all fields in the FinancialData model.

5.4 edgartools for SEC Filings

edgartools provides structured access to SEC EDGAR filings without an API key. For the Risk Agent, it is used to fetch the full text of Item 1A (Risk Factors) from the most recent 10-K filing. This is preferable to web search for U.S.-listed companies because it is the authoritative, unfiltered source for risk disclosures. Tavily web search is used as a fallback for companies that do not file with the SEC (e.g., foreign ADRs).

5.5 Server-Sent Events for Streaming

WebSockets are bidirectional and stateful, making them well-suited for interactive applications such as chat but unnecessarily complex for a one-directional pipeline progress stream. Server-Sent Events are unidirectional (server to client only), require no handshake beyond the initial HTTP request, and are natively supported in browsers via the EventSource API without any client-side library. Since the equity research pipeline is entirely server-driven — the user submits a ticker and waits — SSE is the correct choice.

6. Analysis of Results

6.1 Accuracy and Completeness

The system was tested with four tickers specified in the project brief: AAPL, NVDA, JPM, and COST. In all four cases, the pipeline completed successfully and produced reports with:

- Current price and key valuation metrics (P/E, forward P/E, EPS, 52-week range, analyst target) retrieved correctly.
- Three peer companies identified with appropriate sector and scale matching (e.g., NVDA peers included AMD and AVGO; COST peers included WMT and TGT).
- Five to eight risk factors extracted and presented as specific, actionable statements.
- A BUY, HOLD, or SELL signal with explicit valuation-based reasoning.
- A six-section written report in professional investment research style.

Income statement metrics (revenue, EBITDA) and balance sheet ratios (ROE, D/E, profit margin) are populated for most tickers. Occasional fields may be unavailable when financial data sites do not surface them in Tavily search results for less-covered companies.

6.2 Pipeline Performance

The end-to-end pipeline takes approximately 60 to 120 seconds per research request. The Financial Data Agent (four Tavily searches) completes in roughly 20 to 30 seconds. The Comparable Companies Agent (two to four Tavily searches) completes in 15 to 25 seconds. The Risk Agent (edgartools plus optional Tavily) completes in 10 to 20 seconds. The Analyst Agent (no tools, pure reasoning) completes in 10 to 15 seconds. The Research Writer Agent (no tools, multi-section writing) completes in 30 to 60 seconds.

The streaming UI makes this wait acceptable: each agent's completion is shown to the user as it happens, with a timestamped progress feed, so the experience is interactive rather than a blank-screen wait.

6.3 Signal Quality

Investment signals are reasoned rather than algorithmic. The Analyst Agent is instructed to ground its signal in specific numbers — comparing the subject's P/E to the peer median, assessing the price relative to the 52-week range and analyst target, and explicitly considering risk factors. In testing, the signals produced were directionally consistent with published analyst consensus for the same tickers.

---

7. Broader Implications and Potential Improvements

7.1 Broader Implications

This system demonstrates that a meaningful portion of institutional equity research can be automated at minimal cost. Each research request costs approximately $0.05 to $0.15 in LLM API calls (GPT-4.1-mini at published pricing). A Bloomberg Terminal subscription costs $25,000 per year per seat — for the data alone, without the written synthesis. The gap represents a significant opportunity for smaller investment firms, individual investors, and financial education.

The multi-agent architecture is also broadly applicable. The Researcher-Analyst-Writer pattern is domain-

agnostic: the same pipeline structure could be adapted for legal due diligence (company filings, contract risk, peer litigation), competitive intelligence (market position, competitor moves, analyst commentary), or medical research summarization. The key design principle — specialized agents with typed outputs, sequential dependency, and structured synthesis at the end — transfers across domains.

From a workforce perspective, this system changes the junior analyst's role rather than eliminating it. The agent produces a first draft in 90 seconds; the analyst reviews, challenges, and improves it. The analyst's time is redirected from data gathering to judgment — a net positive for the quality of investment decisions.

7.2 Potential Improvements

Financial data depth: The current implementation retrieves trailing metrics. A more complete system would also retrieve quarterly earnings trends (revenue growth rate, earnings surprise history), forward estimates (consensus revenue and EPS for the next two fiscal years), and dividend history. These are available from financial data APIs such as Financial Modeling Prep (free tier) or Alpha Vantage.

Valuation modelling: The Analyst Agent currently produces a qualitative signal based on comparables. A quantitative improvement would be a discounted cash flow (DCF) model using the retrieved revenue, EBITDA, and growth data. The agent could produce a fair value estimate with a margin of safety, which is a standard component of professional equity research.

Real-time data: Tavily search returns data from financial news articles which may be hours or days old. A premium data source with real-time quotes and live financial data (e.g., Polygon.io's free WebSocket API for real-time prices, or the Financial Modeling Prep real-time quote endpoint) would improve the accuracy of the current price and same-day metrics.

Report export: The current application displays the report in the browser. A production feature would allow exporting the report as a formatted PDF (using a library such as WeasyPrint or Puppeteer) or as a Word document with the firm's research template applied.

Historical comparison: The current system reports trailing twelve-month metrics without historical context. Adding a chart of the company's P/E multiple over the past five years, retrieved from Macrotrends or a similar source, would allow the Analyst Agent to assess whether the current valuation is expensive or cheap relative to history — a standard check in professional research.

Multi-ticker screening: The current interface processes one ticker at a time. A screening mode could accept a list of tickers (e.g., the S&P 500 constituents) and run the pipeline for each, producing a ranked table of signals sorted by strength of conviction. This would enable systematic stock selection.

Confidence and disclaimer calibration: Investment signals carry inherent uncertainty. A future version could have the Analyst Agent produce a confidence level (high, medium, low) alongside the signal, based on the consistency between the quantitative valuation assessment and the qualitative risk assessment. Clearer disclaimers about the AI-generated nature of the analysis should also be surfaced more prominently for any real-world use.

---

8. Conclusion

This capstone project demonstrates the practical application of multi-agent AI systems to a high-value professional workflow. By decomposing equity research into five specialized agents with each agent responsible

for a distinct stage of the analyst's process, the system produces a complete, structured, and reasoned research report in under two minutes. The streaming user interface makes the pipeline transparent and engaging, reflecting the true agent-by-agent progression of the research.

The core technical contributions are:

1) A reliable Tavily-based data retrieval strategy that avoids Yahoo Finance's rate-limiting constraints.

2) A typed Pydantic pipeline that guarantees structured outputs at each stage.

3) A Next.js streaming API route that enables true SSE streaming without buffering.

4) A keepalive mechanism that holds long SSE connections open during extended LLM inference.

The project meets all requirements specified in the capstone brief: it retrieves current financials, identifies comparable companies with valuation multiples, produces a buy/hold/sell signal with reasoning, and displays results in a readable, professional format. It runs successfully in a live demo for all four test tickers.

This report was prepared as part of the Agentic AI Engineering Bootcamp Capstone Project. The equity research reports generated by this application are AI-generated for educational purposes only and do not constitute financial advice.